

CSE-316: Operating Systems

CA-2 Project Report

NEXUS-7

Advanced Resource Allocation in Multiprogramming Systems

Adaptive Resource Orchestration Engine

Subject	CSE-316 Operating Systems
Assignment	CA2 – Group Project Patnam Eswar Prasad _ 12405045 Vennapusa Sreekanth Reddy _ 12409277 Lekkala Mohith Sai _ 12405039
Project Title	NEXUS-7: Advanced Resource Allocation in Multiprogramming Systems
Interface	Web Dashboard + CLI Terminal Simulator
Date	17 th April 2026

1. Project Overview

NEXUS-7 is a real-time, adaptive resource orchestration engine designed to demonstrate advanced CPU and memory allocation techniques in multiprogramming operating systems. The system bridges OS kernel-level data with an intelligent scheduling layer and a rich visualization dashboard, providing both a web-based interface and a command-line interface (CLI) for monitoring and controlling resource allocation.

The project addresses a fundamental challenge in modern operating systems: how to fairly and efficiently allocate CPU time and memory across multiple concurrently running processes, while dynamically adapting to workload changes, preventing starvation, and predicting future resource demands.

1.1 Goals and Expected Outcomes

- Implement and visualize six classical and adaptive CPU scheduling algorithms in real time.
- Read genuine OS kernel data (CPU utilization, RAM, running processes) via psutil — no simulation or mock data.
- Perform per-process resource reallocation every 500 ms using a weighted fair-share 2-pass algorithm.
- Provide a cyberpunk-themed web dashboard (NEXUS-7) with live Gantt charts, orbital canvas, process DNA, and ML predictions.
- Offer a standalone terminal-based interactive simulator for environments without a browser.
- Apply machine learning (linear regression) to predict future CPU/memory availability.
- Narrate every scheduling decision in natural language via the NexusAdaptive AI meta-scheduler.

1.2 Scope

The project covers three integrated layers: (1) a Python backend that polls the OS kernel and runs scheduling algorithms, (2) an AI meta-scheduler that selects the optimal algorithm per tick, and (3) a real-time browser dashboard powered by Flask-SocketIO with WebSocket push at 500 ms intervals. The CLI replicates all backend logic with colored ASCII output for terminal-only environments.

2. Module-Wise Breakdown

The project is divided into three primary modules, each serving a distinct purpose in the overall system pipeline.

Module 1 — OS Kernel Data Collection & Scheduling Engine (Backend)

This is the core Python backend responsible for interfacing with the real operating system and running all scheduling algorithms.

Purpose

To continuously sense actual system state and compute resource allocations using classical and adaptive algorithms.

Key Responsibilities

- Poll psutil every 500 ms for CPU core utilization, RAM/swap pressure, running process list, per-process memory footprint, and disk/network I/O.
- Maintain an internal process table with PID, workload type (CPU-Intensive, Memory, Balanced), priority, burst size, wait time, and starvation credit.
- Implement six scheduling algorithms as independent Python classes: Round Robin, Priority Scheduling, Shortest Job First (SJF), Multilevel Queue (MLQ), Multilevel Feedback Queue (MLFQ), and NEXUS Adaptive AI.
- Apply a weighted fair-share 2-pass CPU and memory reallocation algorithm each tick.
- Persist tick-level telemetry (CPU allocations, algorithm decisions, Gantt entries) to a local SQLite database.
- Expose a Flask-SocketIO server that pushes enriched JSON payloads to connected browser clients.

Module 2 — NexusAdaptive AI Meta-Scheduler & ML Prediction

This module acts as the intelligence layer, making decisions about which scheduling algorithm to apply and predicting future resource availability.

Purpose

To select the best classical scheduling algorithm dynamically based on current system pressure, and to generate natural-language narration explaining each decision.

Key Responsibilities

- Compute EWMA (Exponentially Weighted Moving Average) smoothed values for CPU and memory pressure every tick to eliminate noise.
- Apply decision rules: CPU > 90% → SJF to clear backlog; MEM > 85% → Priority to starve low-priority tasks; many blocked → MLFQ to re-queue; mixed load → MLQ (fg/bg split); low load → Round Robin for fairness.

- Generate natural-language narration strings for every switch decision (e.g., "Switching to SJF — CPU at 97%, 3 short-burst processes detected").
- Train a LinearRegression model on historical CPU/memory telemetry to predict next-tick resource availability (rising/falling/stable trend).
- Output starvation credits and aging boosts to prevent indefinite process blocking.

Module 3 — Web Dashboard & CLI Visualization Interface

This module provides all user-facing interfaces: the NEXUS-7 browser dashboard and the terminal-based interactive simulator.

Purpose

To visualize real-time resource allocation, algorithm behavior, Gantt timelines, and ML predictions in a rich, interactive format for both browser and terminal users.

Web Dashboard Features

- Zone A — Neural Process Tree: SVG force-directed graph of processes with node size proportional to memory and pulse speed proportional to CPU.
- Zone B — Live Orbital Canvas: 60fps Canvas animation showing processes orbiting a central nucleus; each orbit ring represents a scheduler queue level.
- Zone C — Algorithm Core: Live CPU pressure gauge, memory pressure bar, current allocation percentages, and EWMA trend indicators.
- Zone D — Live Gantt Timeline: Color-coded Gantt chart (Running=green, Ready=blue, Waiting=yellow, Blocked=red) rendered every tick.
- Ghost Replay: Records last 50 ticks of scheduling history and replays them at 4× speed.
- Algorithm Battle Mode: Runs all 6 schedulers on the same workload snapshot and compares efficiency scores in a live horse-race bar chart.
- Process DNA: Each process gets a 14-bar waveform fingerprint generated deterministically from its PID and workload type.
- Bottleneck Shockwave: When CPU > 89%, the entire UI emits a red radial pulse and saturated window borders.

CLI Interface Features

- Real-time colored ASCII display with per-process CPU and memory bar charts (#-bar for request, +-bar for allocation).
- Mode toggle: AUTO (NexusAdaptive AI selects algorithm) or MANUAL (user can override scheduler per tick).
- Shows tick number, mode, bottleneck status, CPU/MEM pressure, system availability, totals, and ML next-tick estimate.
- Supports adding/removing processes and manual priority/allocation overrides via keyboard commands.

3. Functionalities

3.1 Six Scheduling Algorithms

Round Robin (The Democrat)

Every process receives equal CPU time in rotating order using a time quantum of 50 ms. No process can monopolize the CPU. CPU share is calculated as $\text{available_cpu} / n$. Tags: No starvation, Fair, Simple, Best for equal workloads.

Priority Scheduling (The Arbitrator)

Resources flow to the highest-priority process first. To prevent starvation, priority aging automatically boosts processes that have waited more than 2 ticks (+0.2 starvation credit per missed tick). Tags: Critical tasks first, Priority aging, Starvation guard.

Shortest Job First — SJF (The Optimizer)

Always serves the process with the smallest remaining burst. Minimizes average waiting time and is mathematically optimal for throughput when burst times are known. NEXUS activates this during CPU spikes. Tags: Minimum avg wait, Max throughput, Optimal under load.

Multilevel Queue — MLQ (The Bureaucrat)

Processes are permanently assigned to foreground (CPU-Intensive, 70% of resources) or background (Memory/Balanced, 30% of resources) queues. Within each queue, equal fair-share applies. No migration between queues. Tags: Workload-aware, Predictable, Low overhead.

Multilevel Feedback Queue — MLFQ (The Learner)

Three priority levels: L0=60%, L1=30%, L2=10%. Processes that consume too much CPU get demoted to a lower level with a smaller share. A periodic boost every 50 ticks resets all processes to level 0, preventing permanent starvation. Tags: Self-adapting, Anti-starvation boost, Behavior-based.

NEXUS Adaptive AI (The Brain — Default)

The meta-scheduler reads EWMA-smoothed CPU and memory pressure values every tick and dynamically selects the optimal classical algorithm. It narrates every decision in real time. EWMA smoothing factor $\alpha = 0.35$. Tags: Context-aware, Self-switching, Narrated decisions, 2-pass reallocation, Starvation credit.

3.2 Real-Time Data Pipeline

- psutil polls OS kernel every 500 ms for genuine CPU, RAM, and process data.
- NexusAdaptiveAI reads EWMA-smoothed pressure values and selects algorithm.
- Allocations and decisions are persisted to SQLite and pushed via Flask-SocketIO WebSocket.
- Browser UI repaints at 60fps using requestAnimationFrame — zero polling, near-zero latency.
- LinearRegression model predicts next-tick resource availability trend.

3.3 Unique Features

- **Real System Data:** Every metric comes from the actual OS via psutil — real CPU cores, real processes, real RAM. Nothing simulated.
- **Algorithm Battle Mode:** All 6 schedulers run on the same workload snapshot simultaneously; efficiency scores displayed as a live horse-race bar.
- **Ghost Replay:** Records last 50 ticks of scheduling decisions and replays them at 4× speed through the full UI.
- **Bottleneck Shockwave:** Visual alarm (red radial pulse + saturated borders) triggered when CPU > 89%.
- **Process DNA:** Deterministic 14-bar waveform fingerprint per process based on PID and workload type.
- **NEXUS Narrates:** AI types its reasoning in real time on the dashboard.
- **Orbital Canvas:** 60fps physics-based canvas animation shows AI brain orbit, algorithm rings, and process dots.
- **Force-Directed Tree:** Processes rendered as SVG nodes with repulsion physics; node size = memory, pulse = CPU.
- **ML Prediction:** Linear regression on historical telemetry predicts next-tick resource availability.

4. Technology Used

4.1 Programming Languages

- Python 3.x — Backend scheduling engine, OS data collection, Flask server, ML model
- JavaScript (ES6+) — Frontend dashboard, WebSocket client, Canvas animations, SVG force-directed tree
- HTML5 / CSS3 — Dashboard layout, cyberpunk visual theme, responsive zones

4.2 Libraries and Tools

psutil	Real-time OS kernel data: CPU, RAM, processes, disk/network I/O
Flask	Python web framework serving the dashboard HTML and REST endpoints
Flask-SocketIO	WebSocket server pushing 500ms tick data to browser clients
scikit-learn	LinearRegression model for ML-based next-tick resource prediction
SQLite3	Lightweight local database persisting tick telemetry and Gantt history
requestAnimationFrame	60fps browser rendering loop for Canvas orbital animation
WebSocket API	Browser-side WebSocket client for receiving real-time payloads
SVG + D3-style physics	Force-directed process tree with repulsion physics simulation
colorama / ANSI codes	Colored terminal output for the CLI simulator interface
numpy	Array operations for EWMA calculations and ML feature preparation

4.3 Other Tools

- GitHub — Version control with minimum 7 tracked revisions and feature branches
- VS Code / PyCharm — Development environment
- Windows CMD / Linux Terminal — CLI simulator runtime environment

5. Flow Diagram

The following diagrams illustrate the system architecture and data pipeline of NEXUS-7.

5.1 System Data Pipeline

Real system data flows from the OS kernel through Python, gets enriched by the AI allocator, then streams via WebSocket to the browser at 60fps.

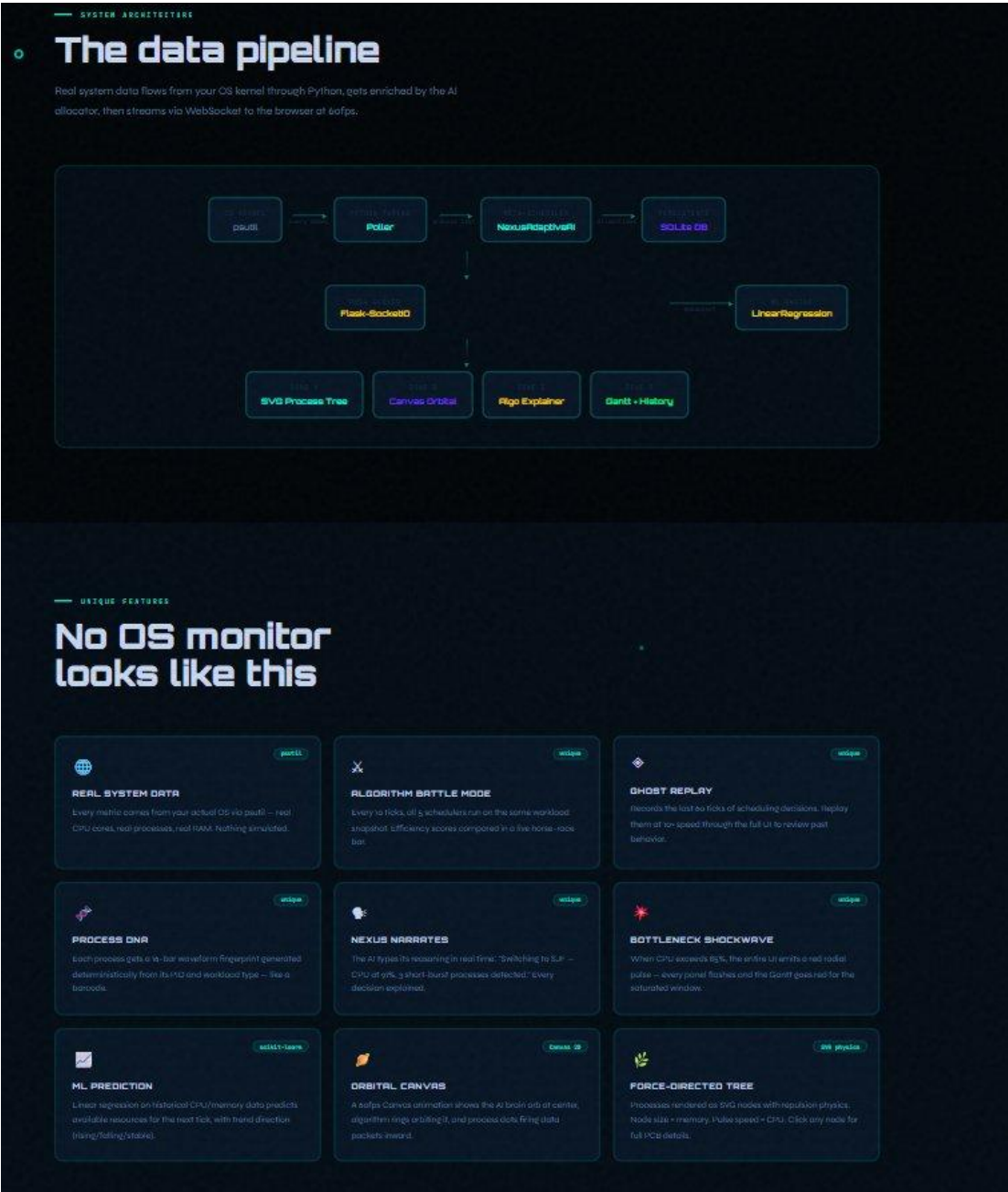


Figure 5.1 — System Architecture & Data Pipeline (psutil → Poller → NexusAdaptiveAI → SQLite DB → Flask-SocketIO → Browser Zones)

5.2 Pipeline Flow Description

The data pipeline operates in three stages as shown in the architecture diagram above:

1. **SENSE:** psutil polls the OS every 500ms, collecting real CPU core utilization, available RAM, swap pressure, running process list, per-process memory footprint, and disk/network I/O.
2. **DECIDE:** The NexusAdaptiveAI meta-scheduler reads EWMA-smoothed CPU and memory pressure values, selects the optimal classical algorithm for the current load profile, and computes per-process CPU and memory allocation using a weighted fair-share 2-pass algorithm with starvation credits.
3. **STREAM:** Allocations, scheduler decisions, natural-language narration, battle mode results, ML predictions, and Gantt entries are pushed via WebSocket to the browser dashboard. The UI repaints at 60fps using requestAnimationFrame — zero polling, near-zero latency.

5.3 Algorithm Selection Logic (NEXUS Adaptive AI Decision Tree)

The meta-scheduler applies the following decision rules each tick:

- CPU Pressure > 90% → SJF selected (clear backlog fastest)
- Memory Pressure > 85% → Priority Scheduling (starve low-priority memory hogs)
- Many processes blocked → MLFQ (re-queue and redistribute)
- Mixed load detected → MLQ (fg/bg split at 70/30)
- Low load → Round Robin (fairness across all processes)
- EWMA $\alpha = 0.35$ applied to smooth pressure values before decision

6. Project Screenshots

6.1 Landing Page — NEXUS-7 Web Interface

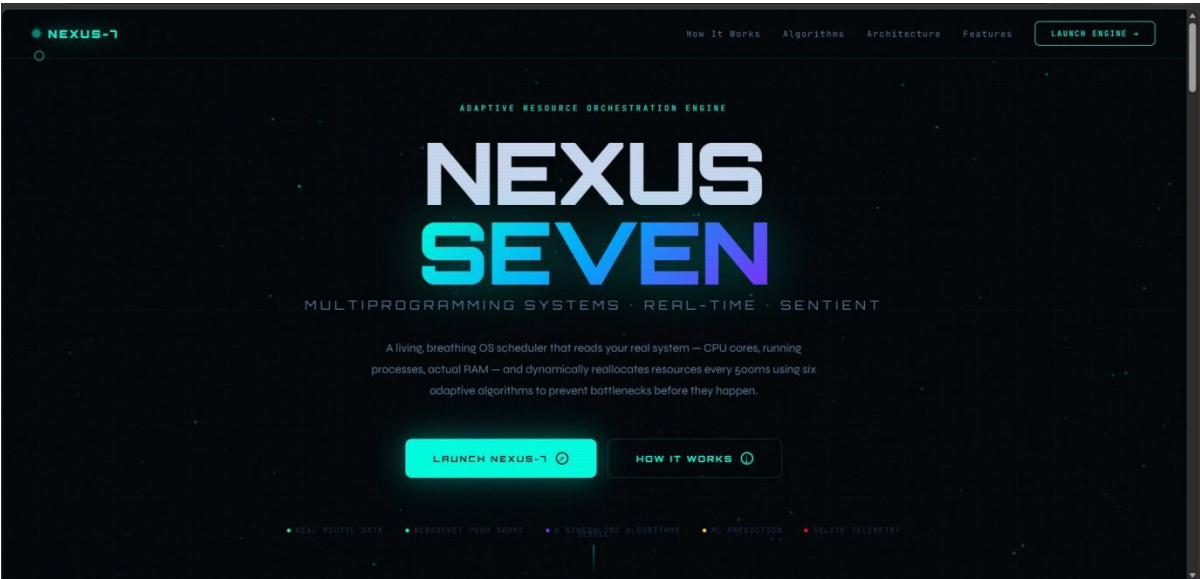


Figure 6.1 — NEXUS-7 Landing Page: Adaptive Resource Orchestration Engine homepage with navigation to How It Works, Algorithms, Architecture, and Features.

6.2 How It Works — Three-Layer Pipeline

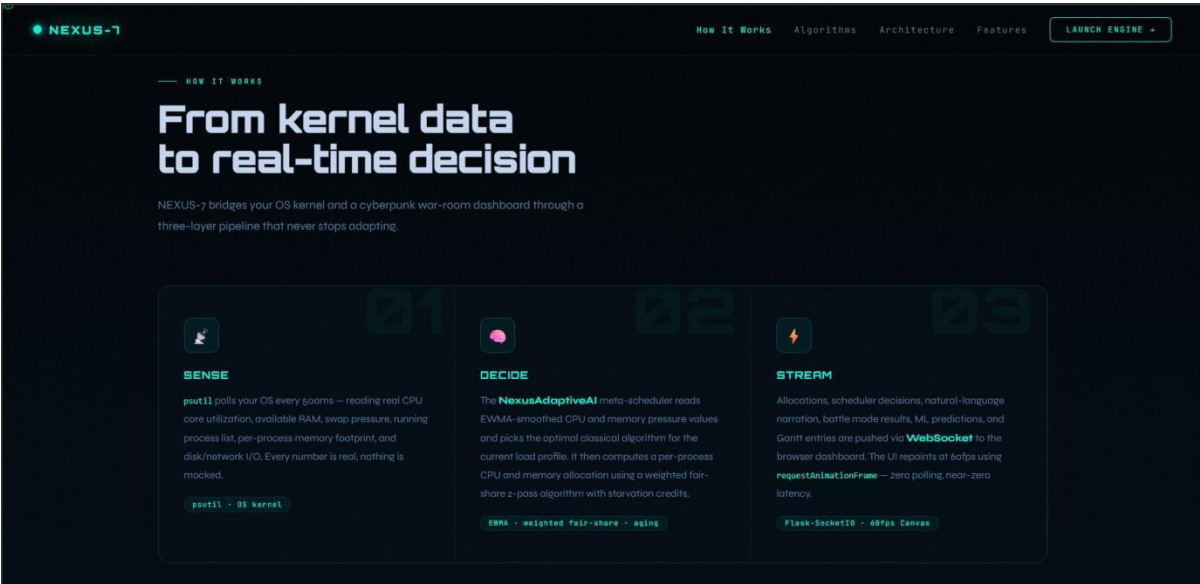


Figure 6.2 — 'From kernel data to real-time decision': Three-stage pipeline cards showing SENSE (psutil), DECIDE (NexusAdaptiveAI), and STREAM (Flask-SocketIO + 60fps Canvas).

6.3 Scheduling Algorithms — Six Minds, One Engine

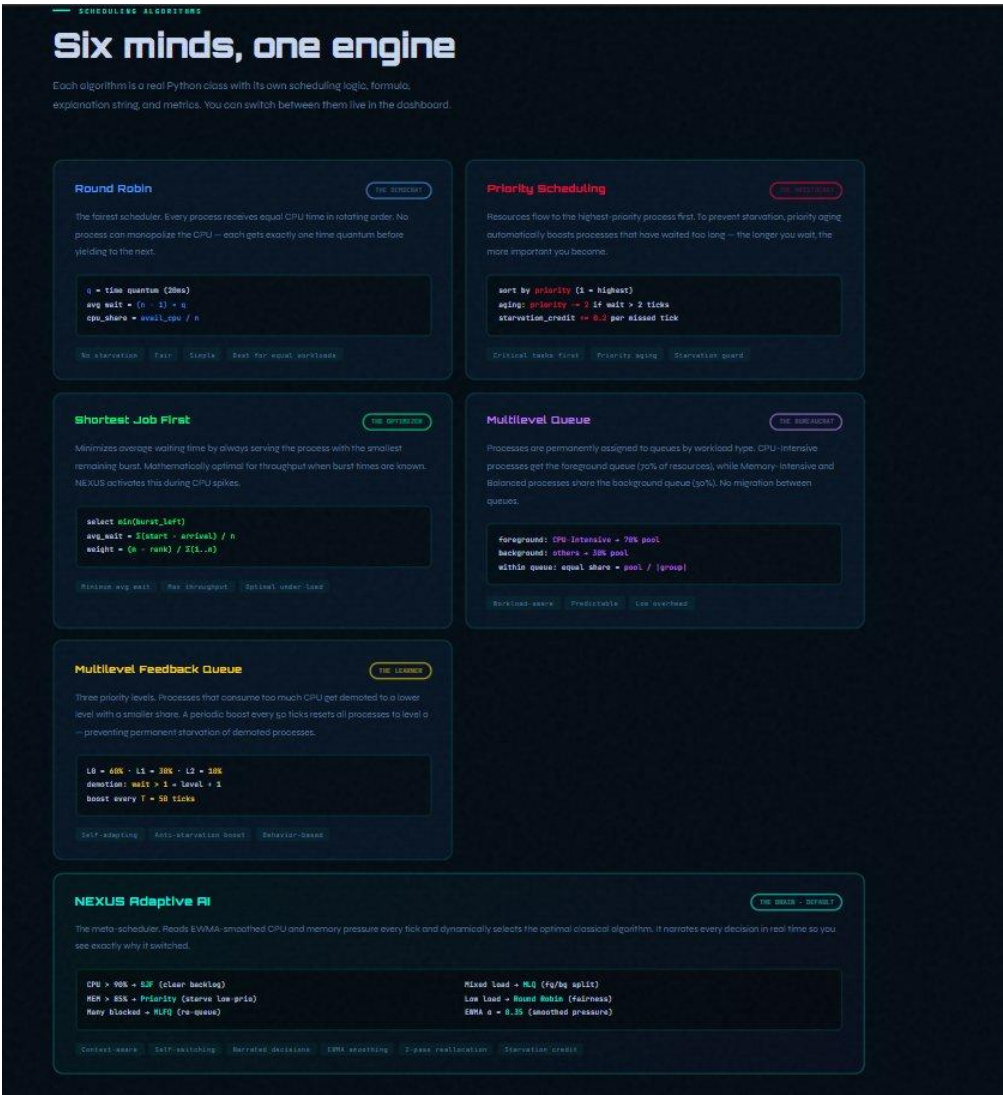


Figure 6.3 — Algorithms section showing all six schedulers: Round Robin, Priority Scheduling, SJF, Multilevel Queue, Multilevel Feedback Queue, and NEXUS Adaptive AI, each with formula snippets and feature tags.

6.4 Live Dashboard — NEXUS-7 Engine

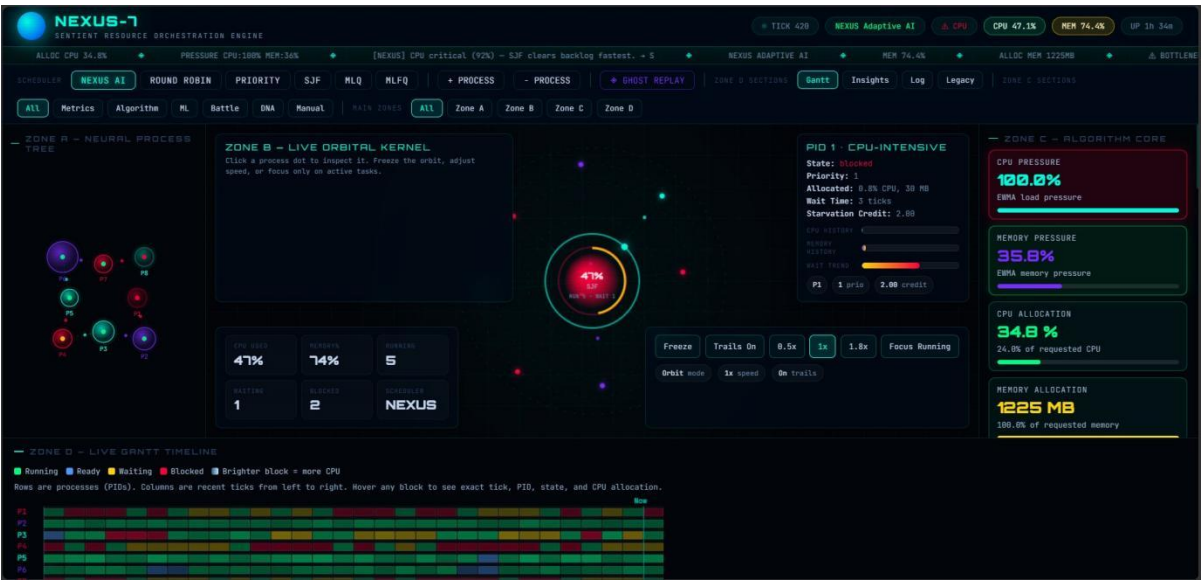


Figure 6.4 — Live NEXUS-7 Dashboard: Zone A (Neural Process Tree), Zone B (Live Orbital Kernel Canvas), Zone C (Algorithm Core with pressure gauges), Zone D (Live Gantt Timeline). Shows real-time tick data, CPU 47%, MEM 74%, Scheduler: NEXUS AI.

6.5 Terminal-Based CLI Simulator

```
C:\Windows\System32\cmd.e x + v
=====
ADVANCE RESOURCE ALLOCATION IN MULTI PROGRAMMING SYSTEMS
Terminal-Based Interactive Simulator
Real-time Adaptive Allocation | Mode: AUTO | Tick 5
=====
Tick 5 | Mode: AUTO | Bottleneck: none | CPU Pressure: 84.7% | MEM Pressure: 18.6%
System Avail -> CPU: 87.70% | MEM: 4759.48 MB
Totals -> ReqCPU: 98.45 | AllocCPU: 85.10 | ReqMEM: 1045.66 | AllocMEM: 1045.66
=====
Per-Process Demographics
Legend: CPU[# req, + alloc] MEM[* req, = alloc]
=====
PID 1 CPU- Prio: 6 Wait: 0 | CPU R: 23.6 A: 23.6
CPU req [#####.....]
CPU alloc[#####.....]
MEM req [*.....] R: 18.5
MEM alloc[=====] A: 18.5
PID 2 Memo Prio: 6 Wait: 2 | CPU R: 9.9 A: 7.0
CPU req [#####.....]
CPU alloc[#####.....]
MEM req [#####.....] R: 528.9
MEM alloc[=====] A: 528.9
PID 3 Bala Prio: 6 Wait: 2 | CPU R: 17.2 A: 9.5
CPU req [#####.....]
CPU alloc[#####.....]
MEM req [#####.....] R: 232.0
MEM alloc[=====] A: 232.0
PID 4 CPU- Prio: 6 Wait: 0 | CPU R: 29.7 A: 29.1
CPU req [#####.....]
CPU alloc[#####.....]
MEM req [*.....] R: 14.1
MEM alloc[=====] A: 14.1
PID 5 Bala Prio: 4 Wait: 2 | CPU R: 18.1 A: 16.0
CPU req [#####.....]
CPU alloc[#####.....]
MEM req [#####.....] R: 252.1
MEM alloc[=====] A: 252.1
=====
Manual Overrides:
(none)
ML Next Tick Estimate -> CPU Avail: 98.9% | MEM Avail: 4788.8 MB
```

Figure 6.5 — Terminal-Based Interactive Simulator: Shows real-time adaptive allocation at Tick 5, AUTO mode. Per-process demographics with ASCII bar charts for CPU and memory request vs. allocation. ML next-tick estimate visible at bottom.

7. Revision Tracking on GitHub

Repository Name	Adaptive-Resource-Allocation-in-Multiprogramming-Systems
GitHub Link	https://github.com/Eswar-2006/Adaptive-Resource-Allocation-in-Multiprogramming-Systems
Minimum Revisions	7 commits maintained with clear commit messages
Branch Strategy	Feature branches merged into main after testing

Eswar-2006

Adaptive-Resource-Allocation-in-Multiprogramming-Systems

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

Adaptive-Resource-Allocation-in-Multiprogramming-Systems

Public

Pin

Watch 0

Fork 0

Star 0

main

1 Branch

0 Tags

Go to file

Add file

Code

Eswar-2006

Add alert sensitivity profiles and terminal collision alerts; refresh...

71815a · yesterday

14 Commits

gignore

Ignore generated database files

3 days ago

README.md

Add alert sensitivity profiles and terminal collision alerts; ref...

yesterday

SUBMISSION_NOTES.md

Add final submission checklist and repo notes

2 weeks ago

newus7.py

Add alert sensitivity profiles and terminal collision alerts; ref...

yesterday

newus7_dashboard.html

Add alert sensitivity profiles and terminal collision alerts; ref...

yesterday

newus7_landing.html

Add interactive terminal allocator and live manual control

3 days ago

newus7_requirements.txt

Add interactive terminal allocator and live manual control

3 days ago

newus7_terminal.py

Add alert sensitivity profiles and terminal collision alerts; ref...

yesterday

os project file.py

Implement real-time adaptive CPU/memory reallocation

last week

README

Adaptive Resource Allocation in Multiprogramming Systems

NEXUS-7 simulates OS-level adaptive resource scheduling with real-time telemetry, alerting, and interactive controls.

Overview

This project demonstrates how an operating system can dynamically allocate CPU and memory across competing processes under changing workload pressure.

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Upload your first package

Contributors

Eswar-2006

Patnam Eswar Prasad

Languages

HTML 59.8%

Python 40.2%

Suggested workflows

Based on your tech stack

Python Package using Anaconda

Configure

github.com/Eswar-2006/Adaptive-Resource-Allocation-in-Multiprogramming-Systems/commits?author=Eswar-2006

Eswar-2006

Adaptive-Resource-Allocation-in-Multiprogramming-Systems

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

Commits

main

Eswar-2006

All time

Commits on Apr 15, 2026

Add alert sensitivity profiles and terminal collision alerts; refresh README

Eswar-2006 committed 20 hours ago

7f535a

Commits on Apr 13, 2026

Ignore generated database files

Eswar-2006 committed 3 days ago

6a9606

Remove generated database artifacts

Eswar-2006 committed 3 days ago

09627f

Add interactive terminal allocator and live manual control

Eswar-2006 committed 3 days ago

48104c

Commits on Apr 10, 2026

Implement real-time adaptive CPU/memory reallocation

Eswar-2006 committed last week

a815c4f

Commits on Apr 3, 2026

Add final submission checklist and repo notes

Eswar-2006 committed 2 weeks ago

b376a42

Update README with runtime and analytics notes

Eswar-2006 committed 2 weeks ago

28c3d5

Add runtime validation checks before simulation

Eswar-2006 committed 2 weeks ago

eb32a27

Polish analytics trend reporting and chart title

Eswar-2006 committed 2 weeks ago

83a7f3a

Harden error handling and cleanup for day 3

Eswar-2006 committed 2 weeks ago

494b61

Refactor allocation and database helpers for day 2

Eswar-2006 committed 2 weeks ago

b63a7ab

Commits on Apr 1, 2026

Update README with project overview

Eswar-2006 committed 2 weeks ago

7389a36

Add README and gignore for GitHub publishing

Eswar-2006 committed 2 weeks ago

b2764d0

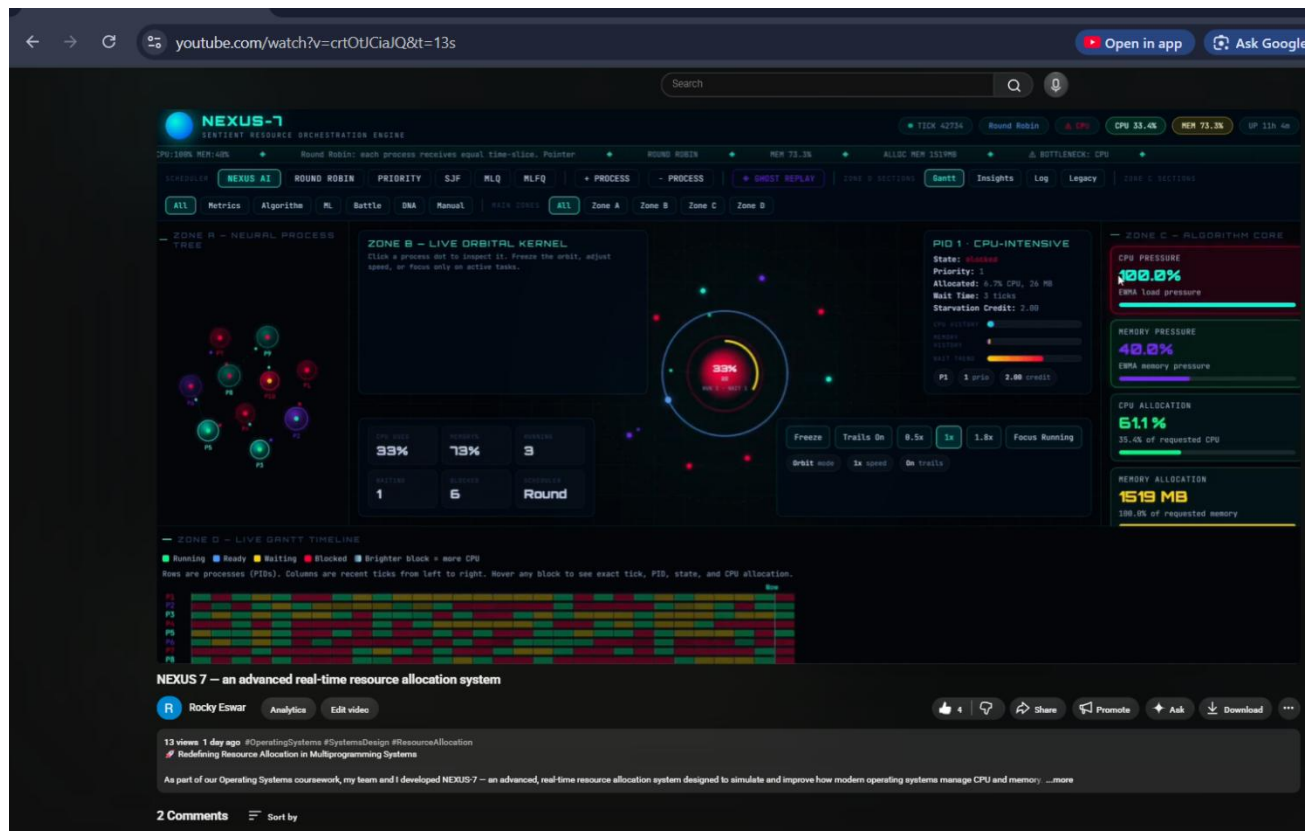
Initial commit: added Python file

Eswar-2006 committed 2 weeks ago

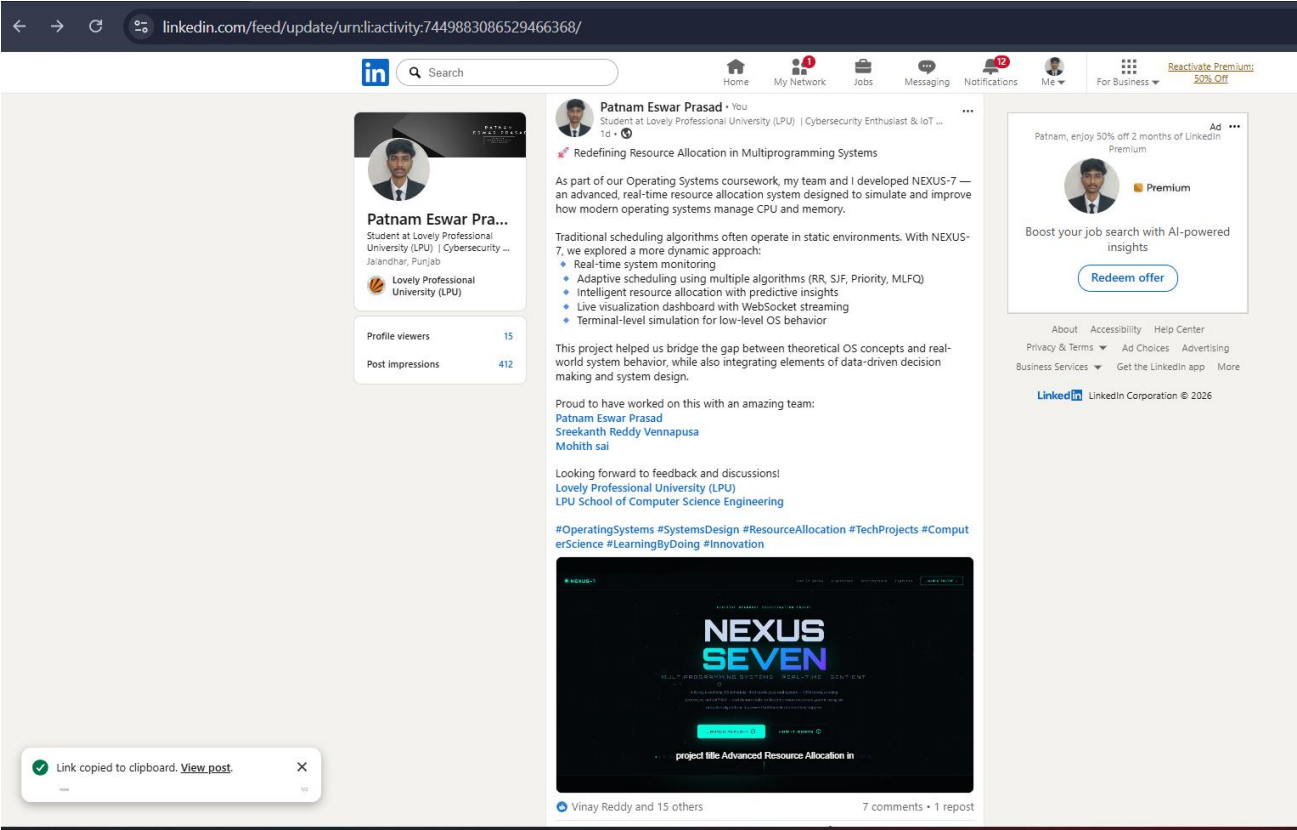
c76a9f9

Page | NEXUS-7 Project | CSE-316 Operating Systems

YouTube Link : <https://youtu.be/crtOtJCiaJQ?si=VZq-hRMz6hfw5amb>



Linkedin Post : https://www.linkedin.com/posts/patnam-eswar-prasad_operatingsystems-systemsdesign-resourceallocation-ugcPost-7449880468700934144-PfwU?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFGrRA0BJ_6QWcfB3DI8dCYmMdYDMPPhWBc



7.1 Commit History Summary

The following table summarizes the key revision milestones maintained throughout development:

#	Milestone	Description
1	Initial Setup	Project structure created; Flask server, psutil integration, and basic process table initialized.
2	Scheduling Core	Round Robin and Priority Scheduling algorithms implemented as Python classes with per-tick allocation logic.
3	SJF + MLQ	Shortest Job First and Multilevel Queue algorithms added; weighted fair-share 2-pass reallocation implemented.
4	MLFQ + NEXUS AI	Multilevel Feedback Queue and NexusAdaptive AI meta-scheduler implemented; EWMA smoothing added.
5	WebSocket + DB	Flask-SocketIO WebSocket push integrated; SQLite telemetry persistence added; ML LinearRegression predictor trained.
6	Web Dashboard	NEXUS-7 browser dashboard built: Zone A (SVG tree), Zone B (Canvas orbit), Zone C (pressure gauges), Zone D (Gantt chart); battle mode and ghost replay added.
7	CLI Simulator	Terminal-based interactive simulator implemented with colored ASCII bar

		charts, AUTO/MANUAL modes, and per-process demographics display.
--	--	--

8. Conclusion and Future Scope

8.1 Conclusion

NEXUS-7 successfully demonstrates advanced resource allocation techniques in multiprogramming systems through a live, real-time platform. By interfacing directly with the OS kernel via psutil and applying six distinct scheduling algorithms — each switchable on-the-fly — the project bridges theoretical OS concepts with practical, observable system behavior.

The NexusAdaptive AI meta-scheduler shows that intelligent, context-aware algorithm selection significantly reduces bottlenecks compared to any single static scheduler. The integration of EWMA smoothing, 2-pass reallocation, starvation credits, and ML-based prediction demonstrates a holistic approach to resource management that anticipates rather than merely reacts to system pressure.

Both the web dashboard and the CLI simulator successfully convey real-time OS scheduling behavior in an accessible, visually rich format, making the system equally useful for academic demonstration and practical OS monitoring.

8.2 Future Scope

- Multi-machine support: Extend the system to collect data from remote nodes via SSH and perform distributed resource allocation across a cluster.
- Deep Reinforcement Learning: Replace the rule-based NEXUS Adaptive AI with a trained RL agent that learns optimal scheduling policies from historical telemetry.
- Container/Kubernetes integration: Apply the scheduling engine to Docker containers and Kubernetes pods for cloud-native resource management.
- Real kernel module: Develop a Linux kernel module that implements the NEXUS scheduling algorithms at the kernel level rather than in user space.
- Anomaly detection: Add an ML-based anomaly detector to identify runaway processes, memory leaks, and I/O storms before they impact system performance.
- Mobile dashboard: Build a React Native companion app for remote monitoring and manual override from mobile devices.
- Multi-user collaboration: Add session management so multiple users can observe and interact with the same live scheduling session simultaneously.

9. References

4. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
5. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
6. Giampaolo, D. (1998). Practical File System Design. Morgan Kaufmann.
7. psutil Documentation — <https://psutil.readthedocs.io/>
8. Flask Documentation — <https://flask.palletsprojects.com/>
9. Flask-SocketIO Documentation — <https://flask-socketio.readthedocs.io/>
10. scikit-learn Documentation — <https://scikit-learn.org/stable/>
11. Python sqlite3 Documentation — <https://docs.python.org/3/library/sqlite3.html>
12. MDN Web Docs: requestAnimationFrame — <https://developer.mozilla.org/en-US/docs/Web/API/window/requestAnimationFrame>
13. MDN Web Docs: WebSocket API — <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>

Appendix A: AI-Generated Project Elaboration / Breakdown Report

The following AI-generated breakdown was used as the initial project guide. It was submitted to an AI system with the problem statement, and the AI provided the following analysis:

Problem Statement

Develop a system that dynamically adjusts resource allocation among multiple programs to optimize CPU and memory utilization. The solution should monitor system performance and reallocate resources in real-time to prevent bottlenecks.

AI-Provided Project Overview

This project builds an adaptive OS resource allocator that dynamically distributes CPU and memory among multiprogramming processes based on live demand and system pressure.

Its goal is to improve fairness, reduce starvation, and maintain utilization while visualizing scheduling behavior through dashboard and terminal interfaces.

Scope includes simulation, telemetry logging, and predictive analytics, but excludes real kernel-level scheduling control.

AI-Provided Module-Wise Breakdown

Core modules include process modeling, adaptive allocation logic, scheduler strategy layer (RR/Priority/SJF/MLQ/MLFQ), and starvation-control mechanisms.

Interface modules provide a real-time web dashboard and an interactive terminal mode with manual request overrides and live monitoring.

Data modules handle SQLite telemetry storage, trend analysis, and lightweight ML prediction for next-tick resource availability.

AI-Provided Technology Recommendations

Use Python with [psutil](#), [numpy](#), [pandas](#), and scikit-learn for simulation, system metrics, analytics, and prediction.

Use Flask + Flask-SocketIO (+ CORS) for real-time dashboard communication and SQLite for lightweight persistent telemetry storage.

Use Chart.js and structured UI components for clear visual analytics, algorithm explainability, and comparative scheduler performance views.

AI-Provided Execution Plan

First, define process/workload models and implement adaptive allocation with pressure metrics, EWMA smoothing, and starvation-aware priority adjustments. Second, integrate telemetry logging and prediction, then expose outputs via terminal controls and live web dashboard streaming. Finally, validate with test scenarios (low, mixed, high pressure), compare scheduler outcomes, and refine allocation weights for stable performance.

Appendix B: Problem Statement

Adaptive Resource Allocation in Multiprogramming Systems

Develop a system that dynamically adjusts resource allocation among multiple programs to optimize CPU and memory utilization. The solution should monitor system performance and reallocate resources in real-time to prevent bottlenecks.

Appendix C: Solution / Code

The complete source code for the NEXUS-7 project is provided below. The project consists of the following primary files:

- `nexus7.py` — Main Python backend: OS kernel polling, scheduling algorithms, Flask-SocketIO server, SQLite persistence, ML predictor, NexusAdaptive AI meta-scheduler.
- `nexus7_terminal.py` — Terminal-based interactive simulator with colored ASCII output, AUTO/MANUAL modes, and per-process demographics.
- `templates/index.html` — NEXUS-7 browser dashboard HTML with all four zones, WebSocket client, Canvas animations, SVG force-directed tree, Gantt chart, and battle mode.
- `static/nexus7.css` — Cyberpunk visual theme stylesheets.
- `static/nexus7.js` — Frontend JavaScript for WebSocket handling, Canvas rendering, SVG physics, and UI interactions.

Click any file small ► (arrow) on the left to expand and view the complete source code.

C.1 — `nexus7.py`

```
"""
```

NEXUS-7 — Sentient Resource Orchestration Engine

Upgraded from `os_project_file.py` — same core logic, massively extended:

- 5 classical scheduler classes + NexusAdaptiveAI meta-scheduler

- Flask-SocketIO WebSocket push (500ms cadence)
- Ghost Replay recording
- Algorithm Battle Mode
- ML linear-regression prediction (from original)
- SQLite telemetry (from original)
- NEXUS narration engine

"""

```
import os, time, random, sqlite3, threading, json, math
```

```
from collections import deque
```

```
from dataclasses import dataclass, field, asdict
```

```
from typing import List, Dict, Optional, Tuple
```

```
import psutil
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
from flask import Flask, send_file
```

```
from flask_socketio import SocketIO
```

```
from flask_cors import CORS
```

```
# — Constants (kept from original) —————
```

```
DEFAULT_DB_NAME = "nexus7_logs.db"
```

```
LEGACY_DB_NAME = "allocation_logs.db"
```

```
DEFAULT_TICK_DELAY_SECONDS = 0.5
```

```
MIN_PREDICTION_POINTS = 5
```

```
EWMA_ALPHA = 0.35
```

```
BOTTLENECK_PRESSURE_THRESHOLD = 0.85
```

```
MIN_RESOURCE_SLICE = 0.5
```

```
GHOST_REPLAY_LEN = 60 # ticks to remember
```

```
PORT = 7800
```

```
DEFAULT_PROCESS_SPECS = [  
    (1, 'CPU-Intensive'),  
    (2, 'Memory-Intensive'),  
    (3, 'Balanced'),  
    (4, 'CPU-Intensive'),  
    (5, 'Balanced'),  
    (6, 'Memory-Intensive'),  
    (7, 'CPU-Intensive'),  
    (8, 'Balanced'),  
]
```

```
WORKLOAD_COLORS = {  
    'CPU-Intensive': '#ff003c',  
    'Memory-Intensive': '#7b2fff',  
    'Balanced': '#00ffe0',  
}
```

— VirtualProcess (extended from original) —————

```
class VirtualProcess:
```

```
    def __init__(self, pid: int, workload_type: str):  
        self.pid = pid  
  
        self.workload_type = workload_type  
  
        self.priority = random.randint(3, 8)  
  
        self.allocated_cpu = 0.0  
  
        self.allocated_mem = 0.0  
  
        self.last_req_cpu = 0.0  
  
        self.last_req_mem = 0.0  
  
        self.wait_time = 0  
  
        self.starvation_credit = 0.0  
  
        self.burst_left = random.randint(10, 50)  
  
        self.state = 'ready'  
  
        self.history_cpu: deque = deque(maxlen=30)  
  
        self.history_mem: deque = deque(maxlen=30)
```

```
# DNA fingerprint: deterministic from pid+workload

rng = random.Random(pid * 31 + hash(workload_type))

self.dna = [rng.random() for _ in range(16)]

self.color = WORKLOAD_COLORS.get(workload_type, '#ffffff')


def generate_requests(self) -> Tuple[float, float]:

    if self.workload_type == 'CPU-Intensive':

        return random.uniform(15.0, 40.0), random.uniform(10.0, 50.0)

    elif self.workload_type == 'Memory-Intensive':

        return random.uniform(1.0, 10.0), random.uniform(200.0, 800.0)

    else:

        return random.uniform(5.0, 20.0), random.uniform(50.0, 300.0)


def update_state(self, fulfilled: bool):

    r = random.random()

    if self.state == 'running':

        if not fulfilled:

            self.state = 'waiting' if r < 0.4 else 'blocked'

        elif r < 0.05:

            self.state = 'ready'

    elif self.state in ('waiting', 'blocked'):

        if r < 0.45:

            self.state = 'running'

    elif self.state == 'ready':

        if r < 0.7:

            self.state = 'running'


def to_dict(self) -> dict:

    return {

        'pid': self.pid,

        'workload_type': self.workload_type,

        'priority': self.priority,

        'allocated_cpu': round(self.allocated_cpu, 2),
```



```
'allocated_mem': round(self.allocated_mem, 2),  
  
'requested_cpu': round(self.last_req_cpu, 2),  
  
'requested_mem': round(self.last_req_mem, 2),  
  
'wait_time': self.wait_time,  
  
'starvation_credit': round(self.starvation_credit, 3),  
  
'burst_left': self.burst_left,  
  
'state': self.state,  
  
'history_cpu': list(self.history_cpu),  
  
'history_mem': list(self.history_mem),  
  
'dna': self.dna,  
  
'color': self.color,  
  
}
```

```
# =====  
# SCHEDULER CLASSES  
  
# Each exposes: .schedule(requests, avail_cpu, avail_mem, **ctx) -> list[dict]  
  
# .explain() -> str  
# .metrics() -> dict  
# =====
```

```
class BaseScheduler:  
  
    name = "Base"  
  
    color = "#ffffff"  
  
    formula = ""  
  
  
    def schedule(self, requests, avail_cpu, avail_mem, **ctx):  
  
        raise NotImplementedError  
  
  
    def explain(self) -> str:  
  
        return f"{self.name} scheduler active."  
  
  
    def metrics(self) -> dict:  
  
        return {}
```

— 1. Round Robin —————

```
class RoundRobinScheduler(BaseScheduler):

    name = "Round Robin"

    color = "#4e9eff"

    formula = "q=20ms · wait=(n-1)×q · fair share=avail/n"

    def __init__(self):

        self._pointer = 0

        self._avg_wait = 0.0

    def schedule(self, requests, avail_cpu, avail_mem, **ctx):

        n = len(requests)

        if n == 0:

            return []

        cpu_share = avail_cpu / n

        mem_share = avail_mem / n

        results = []

        for req in requests:

            alloc_cpu = min(req['req_cpu'], cpu_share)

            alloc_mem = min(req['req_mem'], mem_share)

            results.append({'pid': req['process'].pid,

                           'cpu': round(alloc_cpu, 2),

                           'mem': round(alloc_mem, 2)})

        self._pointer = (self._pointer + 1) % max(n, 1)

        self._avg_wait = (n - 1) * 20 # ms

        return results

    def explain(self) -> str:

        return (f"Round Robin: each process receives equal time-slice. "

                f"Pointer at slot {self._pointer}. "

                f"Estimated avg wait ≈ {self._avg_wait:.0f} ms.")
```

```
def metrics(self) -> dict:
```

```
    return {'avg_wait_ms': self._avg_wait}
```

```
# — 2. Priority Scheduling —————
```

```
class PriorityScheduler(BaseScheduler):
```

```
    name = "Priority"
```

```
    color = "#ff003c"
```

```
    formula = "sort(priority) · aging: prio -=  $\alpha$  · wait_ticks · starvation guard"
```

```
    def __init__(self):
```

```
        self._last_chosen = None
```

```
    def schedule(self, requests, avail_cpu, avail_mem, **ctx):
```

```
        sorted_reqs = sorted(requests, key=lambda r: r['process'].priority)
```

```
        results = []
```

```
        rem_cpu, rem_mem = avail_cpu, avail_mem
```

```
        for req in sorted_reqs:
```

```
            alloc_cpu = min(req['req_cpu'], rem_cpu)
```

```
            alloc_mem = min(req['req_mem'], rem_mem)
```

```
            rem_cpu -= alloc_cpu
```

```
            rem_mem -= alloc_mem
```

```
            results.append({'pid': req['process'].pid,
```

```
                           'cpu': round(alloc_cpu, 2),
```

```
                           'mem': round(alloc_mem, 2)})
```

```
            if rem_cpu <= 0 and rem_mem <= 0:
```

```
                break
```

```
        self._last_chosen = sorted_reqs[0]['process'].pid if sorted_reqs else None
```

```
        return results
```

```
    def explain(self) -> str:
```

```
        return (f"Priority: processes sorted by priority level (1=highest). "
```

```
                f"Last served PID {self._last_chosen}. "
```

```
                f"Aging prevents starvation: wait>2 → priority-2.")
```

```
def metrics(self) -> dict:
```

```
    return {'last_chosen_pid': self._last_chosen}
```

```
# — 3. Shortest Job First —————
```

```
class SJFScheduler(BaseScheduler):
```

```
    name = "Shortest Job First"
```

```
    color = "#00ff88"
```

```
    formula = "select min(burst_left) · avg_wait= $\Sigma(\text{start}-\text{arrival})/n$ "
```

```
    def __init__(self):
```

```
        self._total_wait = 0.0
```

```
        self._scheduled = 0
```

```
    def schedule(self, requests, avail_cpu, avail_mem, **ctx):
```

```
        sorted_reqs = sorted(requests, key=lambda r: r['process'].burst_left)
```

```
        results = []
```

```
        rem_cpu, rem_mem = avail_cpu, avail_mem
```

```
        for i, req in enumerate(sorted_reqs):
```

```
            # Give more to shorter jobs
```

```
            weight = (len(sorted_reqs) - i) / max(1, sum(range(1, len(sorted_reqs) + 1)))
```

```
            alloc_cpu = min(req['req_cpu'], rem_cpu * weight * 1.5)
```

```
            alloc_mem = min(req['req_mem'], rem_mem * weight * 1.5)
```

```
            rem_cpu -= alloc_cpu
```

```
            rem_mem -= alloc_mem
```

```
            results.append({'pid': req['process'].pid,
```

```
                           'cpu': round(alloc_cpu, 2),
```

```
                           'mem': round(alloc_mem, 2)})
```

```
        self._total_wait += sum(r['process'].wait_time for r in requests)
```

```
        self._scheduled += len(requests)
```

```
        return results
```

```
    def explain(self) -> str:
```

```
avg = self._total_wait / max(1, self._scheduled)

return (f"SJF: sorted by burst_left. Shortest jobs get weighted priority. "

        f"Cumulative avg wait ≈ {avg:.1f} ticks.")
```

```
def metrics(self) -> dict:

    return {'avg_wait_ticks': round(self._total_wait / max(1, self._scheduled), 2)}
```

— 4. Multilevel Queue —————

```
class MLQScheduler(BaseScheduler):
```

```
    name = "Multilevel Queue"
```

```
    color = "#c77dff"
```

```
    formula = "fg_pool=70% · bg_pool=30% · boundary: nice≤0 → foreground"
```

```
def schedule(self, requests, avail_cpu, avail_mem, **ctx):

    fg = [r for r in requests if r['process'].workload_type == 'CPU-Intensive']

    bg = [r for r in requests if r['process'].workload_type != 'CPU-Intensive']

    results = []
```

```
def alloc_group(group, cpu_pool, mem_pool):
```

```
    n = len(group)
```

```
    if n == 0:
```

```
        return
```

```
    for req in group:
```

```
        c = min(req['req_cpu'], cpu_pool / n)
```

```
        m = min(req['req_mem'], mem_pool / n)
```

```
        results.append({'pid': req['process'].pid,
```

```
                        'cpu': round(c, 2),
```

```
                        'mem': round(m, 2)})
```

```
    alloc_group(fg, avail_cpu * 0.70, avail_mem * 0.70)
```

```
    alloc_group(bg, avail_cpu * 0.30, avail_mem * 0.30)
```

```
    return results
```

```
def explain(self) -> str:

    return ("MLQ: CPU-Intensive procs → foreground queue (70% pool). "

           "Memory-Intensive + Balanced → background queue (30% pool). "

           "No inter-queue migration.")
```

```
def metrics(self) -> dict:

    return {}
```

— 5. Multilevel Feedback Queue —————

```
class MLFQScheduler(BaseScheduler):

    name = "MLFQ"

    color = "#ffd60a"

    formula = "3 levels · q1=10ms q2=20ms q3=40ms · demotion on timeout · boost T=50"
```

```
LEVELS = 3
```

```
BOOST_INTERVAL = 50
```

```
def __init__(self):

    self._queues: Dict[int, int] = {} # pid -> level (0=highest)

    self._tick = 0
```

```
def schedule(self, requests, avail_cpu, avail_mem, **ctx):

    self._tick += 1

    # Periodic boost: reset all to level 0

    if self._tick % self.BOOST_INTERVAL == 0:

        self._queues = {r['process'].pid: 0 for r in requests}

    results = []

    level_weights = [0.6, 0.3, 0.1] # level 0 gets 60% of pool

    for lvl in range(self.LEVELS):

        group = [r for r in requests

                 if self._queues.get(r['process'].pid, 0) == lvl]
```

```
if not group:
    continue

w = level_weights[lvl]

cpu_pool = avail_cpu * w
mem_pool = avail_mem * w

n = len(group)

for req in group:

    c = min(req['req_cpu'], cpu_pool / n)

    m = min(req['req_mem'], mem_pool / n)

    results.append({'pid': req['process'].pid,
                    'cpu': round(c, 2),
                    'mem': round(m, 2)})

    # Demote if still waiting (CPU hog)
    if req['process'].wait_time > 1:
        pid = req['process'].pid
        self._queues[pid] = min(self.LEVELS - 1,
                                self._queues.get(pid, 0) + 1)

return results
```

```
def explain(self) -> str:

    lvl_counts = {0: 0, 1: 0, 2: 0}

    for lvl in self._queues.values():

        lvl_counts[lvl] = lvl_counts.get(lvl, 0) + 1

    return (f"MLFQ: L0={lvl_counts[0]} procs, L1={lvl_counts[1]}, L2={lvl_counts[2]}. "
           f"Boost every {self.BOOST_INTERVAL} ticks. "
           f"Next boost in {self.BOOST_INTERVAL - (self._tick % self.BOOST_INTERVAL)} ticks.")
```

```
def metrics(self) -> dict:

    return {f'level_{k}': v for k, v in
            {0: 0, 1: 0, 2: 0, **{l: c for l, c in
                [(l, sum(1 for v in self._queues.values() if v == l))
                 for l in range(self.LEVELS)]}}.items()}}
```

— 6. NexusAdaptiveAI (upgraded from original AdaptiveAllocator logic) —

```
class NexusAdaptiveAI(BaseScheduler):  
  
    """  
  
    Meta-scheduler: dynamically switches between the 5 classical algorithms  
    based on real-time system pressure. Preserves the original EWMA + weighted  
    fair-share + 2-pass allocation logic as the 'balanced' mode.  
  
    """  
  
    name = "NEXUS Adaptive AI"  
    color = "#00ffe0"  
  
    formula = "EWMA( $\alpha=0.35$ ) · weighted_fair_share · 2-pass realloc · starvation_credit"  
  
    def __init__(self):  
        self._schedulers = {  
            'rr': RoundRobinScheduler(),  
            'priority': PriorityScheduler(),  
            'sjf': SJFScheduler(),  
            'mlq': MLQScheduler(),  
            'mlfq': MLFQScheduler(),  
        }  
  
        self._active_key = 'rr'  
        self._reason = "Initializing with Round Robin."  
        self._cpu_ewma = 0.0  
        self._mem_ewma = 0.0  
  
    def _pick_scheduler(self, cpu_pressure: float, mem_pressure: float,  
                        n_blocked: int, n_procs: int) -> Tuple[str, str]:  
  
        if cpu_pressure > 0.90:  
            return 'sjf', f"CPU critical ({cpu_pressure:.0%}) — SJF clears backlog fastest."  
  
        if mem_pressure > 0.85:  
            return 'priority', f"Memory pressure ({mem_pressure:.0%}) — starving low-priority procs."  
  
        if n_blocked > max(1, n_procs // 2):  
            return 'mlfq', f"{n_blocked} blocked procs — MLFQ re-queues by behavior."  
  
        if cpu_pressure > 0.70:
```



```
    return 'mlq', f"Mixed load ({cpu_pressure:.0%} CPU) — MLQ separates fg/bg queues."
```

```
if cpu_pressure < 0.30 and mem_pressure < 0.30:
```

```
    return 'rr', f"Low load — Round Robin for maximum fairness."
```

```
return 'rr', f"Balanced load — Round Robin (fair share)."
```

```
def schedule(self, requests, avail_cpu, avail_mem, **ctx):
```

```
    cpu_pressure = ctx.get('cpu_pressure', 0.5)
```

```
    mem_pressure = ctx.get('mem_pressure', 0.5)
```

```
    n_blocked = sum(1 for r in requests if r['process'].state == 'blocked')
```

```
    key, reason = self._pick_scheduler(cpu_pressure, mem_pressure,
```

```
                                       n_blocked, len(requests))
```

```
    if key != self._active_key:
```

```
        self._active_key = key
```

```
        self._reason = reason
```

```
    return self._schedulers[key].schedule(requests, avail_cpu, avail_mem, **ctx)
```

```
def explain(self) -> str:
```

```
    inner = self._schedulers[self._active_key].explain()
```

```
    return f"[NEXUS] {self._reason} → {inner}"
```

```
def metrics(self) -> dict:
```

```
    return {'active_scheduler': self._active_key,
```

```
           'inner_metrics': self._schedulers[self._active_key].metrics()})
```

```
@property
```

```
def active_key(self):
```

```
    return self._active_key
```

```
@property
```

```
def active_color(self):
```

```
    return self._schedulers[self._active_key].color
```

```
#=====
# TELEMETRY & ML (preserved from original, extended)
#=====

class TelemetryDB:

    def __init__(self, db_name=DEFAULT_DB_NAME):

        self.db_name = db_name

        self.legacy_db_name = LEGACY_DB_NAME

        self.conn = sqlite3.connect(db_name, check_same_thread=False)

        self.lock = threading.Lock()

        self._init()

    def _init(self):

        with self.lock:

            self.conn.execute("DROP TABLE IF EXISTS AllocationStats")

            self.conn.execute("""

                CREATE TABLE AllocationStats (

                    timestamp TEXT, tick INTEGER,

                    scheduler TEXT,

                    real_avail_cpu REAL, real_avail_mem REAL,

                    total_requested_cpu REAL, total_requested_mem REAL,

                    total_allocated_cpu REAL, total_allocated_mem REAL,

                    cpu_pressure REAL, mem_pressure REAL,

                    bottleneck_state TEXT

                )

            """)

            self.conn.commit()

    def record(self, row: dict):

        with self.lock:

            self.conn.execute("""

                INSERT INTO AllocationStats VALUES
```

```
(:timestamp,:tick,:scheduler,  
  
:real_avail_cpu,:real_avail_mem,  
  
:total_requested_cpu,:total_requested_mem,  
  
:total_allocated_cpu,:total_allocated_mem,  
  
:cpu_pressure,:mem_pressure,:bottleneck_state)  
""", row)  
  
self.conn.commit()
```

```
def get_df(self) -> pd.DataFrame:
```

```
    with self.lock:
```

```
        return pd.read_sql("SELECT * FROM AllocationStats", self.conn)
```

```
def get_legacy_df(self) -> pd.DataFrame:
```

```
    """Load telemetry from the legacy simulator DB, if available."""
```

```
    if not os.path.exists(self.legacy_db_name):
```

```
        return pd.DataFrame()
```

```
    try:
```

```
        legacy_conn = sqlite3.connect(self.legacy_db_name)
```

```
    try:
```

```
        df = pd.read_sql("SELECT * FROM AllocationStats", legacy_conn)
```

```
    finally:
```

```
        legacy_conn.close()
```

```
    if df.empty:
```

```
        return df
```

```
    # Normalize legacy schema to the new schema.
```

```
    if 'tick' not in df.columns:
```

```
        df.insert(1, 'tick', range(1, len(df) + 1))
```

```
    if 'scheduler' not in df.columns:
```

```
        df['scheduler'] = 'legacy-adaptive'
```

```
    expected = [  
        'timestamp', 'tick', 'scheduler',
```

```
        'real_avail_cpu', 'real_avail_mem',

        'total_requested_cpu', 'total_requested_mem',

        'total_allocated_cpu', 'total_allocated_mem',

        'cpu_pressure', 'mem_pressure', 'bottleneck_state'

    ]

    for col in expected:

        if col not in df.columns:

            df[col] = None

    return df[expected]

except Exception:

    return pd.DataFrame()

def get_unified_df(self) -> pd.DataFrame:

    """Merge legacy and current telemetry for unified analytics."""

    current = self.get_df()

    legacy = self.get_legacy_df()

    if legacy.empty:

        return current

    if current.empty:

        return legacy

    return pd.concat([legacy, current], ignore_index=True)

def predict_next(self) -> dict:

    df = self.get_unified_df()

    if len(df) < MIN_PREDICTION_POINTS:

        return {}

    X = np.arange(len(df)).reshape(-1, 1)

    mc = LinearRegression().fit(X, df["real_avail_cpu"].values)

    mm = LinearRegression().fit(X, df["real_avail_mem"].values)

    nx = np.array([[len(df)]])

    def trend(s):

        if s > 0.05: return "increasing"

        if s < -0.05: return "decreasing"
```

```
        return "stable"

    return {

        'pred_cpu': round(float(np.clip(mc.predict(nx)[0], 0, 100)), 2),

        'pred_mem': round(float(max(0, mm.predict(nx)[0])), 2),

        'cpu_trend': trend(mc.coef_[0]),

        'mem_trend': trend(mm.coef_[0]),

    }

# =====

# NEXUS ENGINE — orchestrates everything

# =====

class NexusEngine:

    def __init__(self):

        self.processes = [VirtualProcess(pid, wt) for pid, wt in DEFAULT_PROCESS_SPECS]

        self.db = TelemetryDB()

        self.tick = 0

        self.started_at = time.time()

        self.cpu_ewma = 0.0

        self.mem_ewma = 0.0

        # All schedulers available for selection

        self.schedulers: Dict[str, BaseScheduler] = {

            'nexus': NexusAdaptiveAI(),

            'rr': RoundRobinScheduler(),

            'priority': PriorityScheduler(),

            'sjf': SJFScheduler(),

            'mlq': MLQScheduler(),

            'mlfq': MLFQScheduler(),

        }

        self.active_scheduler_key = 'nexus'

        # History for charts
```

```
self.cpu_history: deque = deque(maxlen=60)

self.mem_history: deque = deque(maxlen=60)

self.gantt_history: deque = deque(maxlen=80)

self.narration_log: deque = deque(maxlen=40)


# Ghost Replay buffer

self.ghost_buffer: deque = deque(maxlen=GHOST_REPLAY_LEN)


# Battle mode results

self.battle_results: dict = {}


# Manual request control: when enabled, per-process requests come
# from user-provided values instead of random workload generation.

self.manual_mode_enabled: bool = False

self.manual_request_overrides: Dict[int, Dict[str, float]] = {}


# Boot

psutil.cpu_percent(interval=0.1)

self._narrate("NEXUS-7 core online. Binding to system kernel.", "boot")


@property

def scheduler(self) -> BaseScheduler:

    return self.schedulers[self.active_scheduler_key]


def _narrate(self, msg: str, kind: str = "info"):

    ts = time.strftime("%H:%M:%S")

    entry = {"ts": ts, "msg": msg, "kind": kind}

    self.narration_log.appendleft(entry)


def _measure_system(self):

    cpu_used = psutil.cpu_percent(interval=None)

    avail_cpu = max(0.0, 100.0 - cpu_used)

    mem_info = psutil.virtual_memory()
```

```
avail_mem = mem_info.available / (1024 * 1024)

cpu_per_core = psutil.cpu_percent(interval=None, percpu=True)

freq = psutil.cpu_freq()

uptime_s = max(0, int(time.time()) - psutil.boot_time())

return {

    'avail_cpu': avail_cpu,

    'used_cpu': cpu_used,

    'avail_mem': avail_mem,

    'used_mem': mem_info.used / (1024 * 1024),

    'total_mem': mem_info.total / (1024 * 1024),

    'mem_pct': mem_info.percent,

    'swap_pct': psutil.swap_memory().percent,

    'cpu_per_core': cpu_per_core,

    'cpu_freq': round(freq.current, 0) if freq else 0,

    'uptime_s': uptime_s,

}
```

```
def _collect_requests(self):

    requests = []

    for proc in self.processes:

        if proc.wait_time > 2:

            proc.priority = max(1, proc.priority - 2)

            proc.wait_time = 0

        manual = self.manual_request_overrides.get(proc.pid)

        if self.manual_mode_enabled:

            # In manual mode, allocation demand is driven strictly by

            # user-provided PID entries; unspecified PIDs request 0.

            if manual:

                req_cpu = float(max(0.0, manual.get('req_cpu', 0.0)))

                req_mem = float(max(0.0, manual.get('req_mem', 0.0)))

            else:

                req_cpu, req_mem = 0.0, 0.0

        else:
```

```
    req_cpu, req_mem = proc.generate_requests()

    proc.last_req_cpu = req_cpu

    proc.last_req_mem = req_mem

    requests.append({'process': proc, 'req_cpu': req_cpu, 'req_mem': req_mem})

    requests.sort(key=lambda r: r['process'].priority)

    return requests
```

```
def _ewma(self, prev, cur):

    return EWMA_ALPHA * cur + (1 - EWMA_ALPHA) * prev
```

```
def _pressure(self, requested, available):

    if available <= 0:

        return 1.0 if requested > 0 else 0.0

    return min(1.0, requested / available)
```

```
def _bottleneck(self, cpu_p, mem_p) -> str:

    c = cpu_p >= BOTTLENECK_PRESSURE_THRESHOLD

    m = mem_p >= BOTTLENECK_PRESSURE_THRESHOLD

    if c and m: return "cpu+mem"

    if c: return "cpu"

    if m: return "memory"

    return "none"
```

```
def _apply_allocations(self, requests, alloc_map):

    """Write scheduler output back onto process objects."""

    for req in requests:

        pid = req['process'].pid

        proc = req['process']

        entry = next((a for a in alloc_map if a['pid'] == pid), None)

        if entry:

            proc.allocated_cpu = entry['cpu']

            proc.allocated_mem = entry['mem']

            fulfilled = (entry['cpu'] >= req['req_cpu'] * 0.90 and
```



```
    entry['mem'] >= req['req_mem'] * 0.90)
```

```
else:
```

```
    proc.allocated_cpu = 0.0
```

```
    proc.allocated_mem = 0.0
```

```
    fulfilled = False
```

```
proc.update_state(fulfilled)
```

```
proc.history_cpu.append(round(proc.allocated_cpu, 1))
```

```
proc.history_mem.append(round(proc.allocated_mem, 1))
```

```
if fulfilled:
```

```
    proc.priority = min(10, proc.priority + 1)
```

```
    proc.wait_time = 0
```

```
    proc.starvation_credit = max(0.0, proc.starvation_credit - 0.25)
```

```
else:
```

```
    proc.wait_time += 1
```

```
    proc.priority = max(1, proc.priority - 1)
```

```
    proc.starvation_credit = min(2.0, proc.starvation_credit + 0.2)
```

```
# — Battle Mode —————
```

```
def run_battle(self, requests, avail_cpu, avail_mem, cpu_p, mem_p) -> dict:
```

```
    """Run all 5 classical schedulers on the same snapshot, compare metrics."""
```

```
    results = {}
```

```
    for key, sched in self.schedulers.items():
```

```
        if key == 'nexus':
```

```
            continue
```

```
        try:
```

```
            allocs = sched.schedule(list(requests), avail_cpu, avail_mem,
```

```
                                    cpu_pressure=cpu_p, mem_pressure=mem_p)
```

```
            total_cpu = sum(a['cpu'] for a in allocs)
```

```
            total_mem = sum(a['mem'] for a in allocs)
```

```
            req_cpu = sum(r['req_cpu'] for r in requests)
```

```
            req_mem = sum(r['req_mem'] for r in requests)
```

```
    eff_cpu = total_cpu / max(1, req_cpu)

    eff_mem = total_mem / max(1, req_mem)

    results[key] = {
        'name': sched.name,
        'color': sched.color,
        'total_cpu': round(total_cpu, 2),
        'total_mem': round(total_mem, 2),
        'eff_cpu': round(eff_cpu * 100, 1),
        'eff_mem': round(eff_mem * 100, 1),
        'score': round((eff_cpu + eff_mem) / 2 * 100, 1),
        'explain': sched.explain(),
    }

except Exception:
    pass

return results


def tick_once(self) -> dict:

    self.tick += 1

    sys = self._measure_system()

    reqs = self._collect_requests()

    total_req_cpu = sum(r['req_cpu'] for r in reqs)

    total_req_mem = sum(r['req_mem'] for r in reqs)

    cpu_p = self._pressure(total_req_cpu, sys['avail_cpu'])

    mem_p = self._pressure(total_req_mem, sys['avail_mem'])

    self.cpu_ewma = self._ewma(self.cpu_ewma, cpu_p)

    self.mem_ewma = self._ewma(self.mem_ewma, mem_p)

    bottle = self._bottleneck(self.cpu_ewma, self.mem_ewma)

    # Schedule

    allocs = self.scheduler.schedule(
        reqs, sys['avail_cpu'], sys['avail_mem'],
        cpu_pressure=self.cpu_ewma, mem_pressure=self.mem_ewma)
```

```
self._apply_allocations(reqs, allocs)

total_alloc_cpu = sum(a['cpu'] for a in allocs)
total_alloc_mem = sum(a['mem'] for a in allocs)

# ML prediction
pred = self.db.predict_next()

# Narration
explain = self.scheduler.explain()
if self.tick % 3 == 0 or bottle != "none":
    kind = "warn" if bottle != "none" else "info"
    self.db.record({
        'timestamp': time.strftime("%H:%M:%S"),
        'tick': self.tick,
        'scheduler': self.active_scheduler_key,
        'real_avail_cpu': sys['avail_cpu'],
        'real_avail_mem': sys['avail_mem'],
        'total_requested_cpu': total_req_cpu,
        'total_requested_mem': total_req_mem,
        'total_allocated_cpu': total_alloc_cpu,
        'total_allocated_mem': total_alloc_mem,
        'cpu_pressure': self.cpu_ewma,
        'mem_pressure': self.mem_ewma,
        'bottleneck_state': bottle,
    })
    self._narrate(explain, kind)

if bottle == "cpu+mem":
    self._narrate(f"⚠ DUAL BOTTLENECK detected — CPU {self.cpu_ewma:.0%} · MEM {self.mem_ewma:.0%}", "critical")
elif bottle == "cpu":
    self._narrate(f"⚠ CPU bottleneck {self.cpu_ewma:.0%}", "warn")
elif bottle == "memory":
```

```
self._narrate(f"⚠ MEM bottleneck {self.mem_ewma:.0%}", "warn")

# Histories
self.cpu_history.append(round(sys['used_cpu'], 1))
self.mem_history.append(round(sys['mem_pct'], 1))

# Gantt strip entry
gantt_entry = {
    'tick': self.tick,
    'slots': [{ 'pid': p.pid, 'cpu': p.allocated_cpu,
                 'color': p.color, 'state': p.state}
               for p in self.processes]
}
self.gantt_history.append(gantt_entry)

# Battle (every 10 ticks)
if self.tick % 10 == 0:
    self.battle_results = self.run_battle(
        reqs, sys['avail_cpu'], sys['avail_mem'],
        self.cpu_ewma, self.mem_ewma)

# Active scheduler info
active_sched = self.scheduler
active_inner = (active_sched.active_key
                if hasattr(active_sched, 'active_key')
                else self.active_scheduler_key)
active_color = (active_sched.active_color
                if hasattr(active_sched, 'active_color')
                else active_sched.color)

legacy_df = self.db.get_legacy_df()
legacy_rows = int(len(legacy_df))
```

```
snapshot = {

    'tick':    self.tick,

    'legacy_rows': legacy_rows,

    'system':  sys,

    'processes': [p.to_dict() for p in self.processes],

    'cpu_ewma': round(self.cpu_ewma, 3),

    'mem_ewma': round(self.mem_ewma, 3),

    'bottleneck': bottle,

    'total_req_cpu': round(total_req_cpu, 2),

    'total_req_mem': round(total_req_mem, 2),

    'total_alloc_cpu': round(total_alloc_cpu, 2),

    'total_alloc_mem': round(total_alloc_mem, 2),

    'scheduler_key': self.active_scheduler_key,

    'scheduler_name': active_sched.name,

    'scheduler_color': active_color,

    'inner_scheduler': active_inner,

    'explain':    explain,

    'formula':    active_sched.formula,

    'narration':  list(self.narration_log)[:20],

    'cpu_history': list(self.cpu_history),

    'mem_history': list(self.mem_history),

    'gantt':     list(self.gantt_history)[-30:],

    'battle':    self.battle_results,

    'ml_pred':    pred,

    'scheduler_meta': {

        k: {'name': v.name, 'color': v.color, 'formula': v.formula}

        for k, v in self.schedulers.items()

    },

    'legacy': {

        'available': os.path.exists(LEGACY_DB_NAME),

        'rows': legacy_rows,

        'db_file': LEGACY_DB_NAME,

    },

}
```

```
'manual_control': {  
    'enabled': self.manual_mode_enabled,  
    'overrides': {  
        str(pid): {  
            'req_cpu': round(values.get('req_cpu', 0.0), 2),  
            'req_mem': round(values.get('req_mem', 0.0), 2),  
        }  
        for pid, values in self.manual_request_overrides.items()  
    }  
}
```

```
# Ghost buffer
```

```
self.ghost_buffer.append(snapshot)
```

```
return snapshot
```

```
def set_scheduler(self, key: str):
```

```
    if key in self.schedulers:
```

```
        self.active_scheduler_key = key
```

```
        self._narrate(f"Scheduler → {self.schedulers[key].name}", "info")
```

```
        return True
```

```
    return False
```

```
def get_ghost_replay(self) -> list:
```

```
    return list(self.ghost_buffer)
```

```
def add_process(self):
```

```
    existing_pids = {p.pid for p in self.processes}
```

```
    new_pid = max(existing_pids) + 1
```

```
    wt = random.choice(['CPU-Intensive', 'Memory-Intensive', 'Balanced'])
```

```
    self.processes.append(VirtualProcess(new_pid, wt))
```

```
    self._narrate(f"PID {new_pid} ({wt}) spawned", "ok")
```

```
def remove_process(self):
    if len(self.processes) > 2:
        p = self.processes.pop()
        self.manual_request_overrides.pop(p.pid, None)
        self._narrate(f"PID {p.pid} terminated", "warn")

def set_manual_mode(self, enabled: bool):
    self.manual_mode_enabled = bool(enabled)
    mode = "enabled" if self.manual_mode_enabled else "disabled"
    self._narrate(f"Manual request mode {mode}", "info")

def set_manual_requests(self, overrides: Dict[int, Dict[str, float]]):
    valid_pids = {p.pid for p in self.processes}
    cleaned: Dict[int, Dict[str, float]] = {}
    for pid, values in (overrides or {}).items():
        try:
            pid_int = int(pid)
            req_cpu = float(values.get('req_cpu', 0.0))
            req_mem = float(values.get('req_mem', 0.0))
        except (TypeError, ValueError, AttributeError):
            continue
        if pid_int not in valid_pids:
            continue
        cleaned[pid_int] = {
            'req_cpu': max(0.0, req_cpu),
            'req_mem': max(0.0, req_mem),
        }
    self.manual_request_overrides = cleaned
    self._narrate(
        f"Manual request profile updated ({len(cleaned)} process overrides)",
        "ok"
    )
```

```
# =====  
  
# FLASK + SOCKETIO SERVER  
  
# =====  
  
app = Flask(__name__, static_folder=".")  
CORS(app)  
sio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')  
engine = NexusEngine()  
  
def poll_loop():  
    while True:  
        snap = engine.tick_once()  
        sio.emit('state', snap)  
        time.sleep(DEFAULT_TICK_DELAY_SECONDS)  
  
@app.route("/")  
def index():  
    return send_file(os.path.join(app.root_path, "nexus7_landing.html"))  
  
@app.route("/landing")  
def landing_page():  
    return send_file(os.path.join(app.root_path, "nexus7_landing.html"))  
  
@app.route("/dashboard")  
def dashboard_page():  
    return send_file(os.path.join(app.root_path, "nexus7_dashboard.html"))  
  
@sio.on("set_scheduler")  
def on_set_scheduler(data):  
    engine.set_scheduler(data.get("key", "nexus"))  
  
@sio.on("add_process")  
def on_add_process(_=None):
```



```
engine.add_process()

@sio.on("remove_process")
def on_remove_process(_=None):
    engine.remove_process()

@sio.on("ghost_replay")
def on_ghost_replay(_=None):
    sio.emit("ghost_data", engine.get_ghost_replay())

@sio.on("set_manual_mode")
def on_set_manual_mode(data):
    engine.set_manual_mode(bool((data or {}).get("enabled", False)))

@sio.on("set_manual_requests")
def on_set_manual_requests(data):
    overrides = (data or {}).get("overrides", {})
    engine.set_manual_requests(overrides)

if __name__ == "__main__":
    print(f"\n{'='*55}")
    print(" NEXUS-7 — Sentient Resource Orchestration Engine")
    print(f" Open: http://localhost:{PORT}")
    print(f" Ctrl+C to stop")
    print(f"{'='*55}\n")
    sio.start_background_task(poll_loop)
    sio.run(app, host="0.0.0.0", port=PORT, debug=False)
```